A Project Report

On

BLOG PUBLISHING PLATFORM

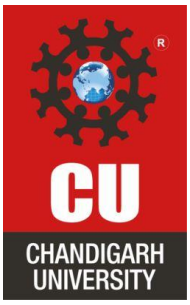
(Inkly – A Community Blogging Website)

Submitted in partial fulfillment of the requirement for the award of degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



FULL STACK – II (24CSP – 339)

Submitted by

Renulasya (24BAI70210)

Submitted to

Ms. Lovleen Kaur (E20454)

APEX INSTITUTE OF TECHNOLOGY, CSE – AIML

CHANDIGARH UNIVERSITY, GHARUAN

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of this project.

First, I extend my heartfelt thanks to my project mentor, Ms. Lovleen Kaur, for her valuable guidance, continuous support, encouragement, and constructive feedback. I am also thankful to the faculty members of Apex Institute of Technology (CSE-AIML), Chandigarh University, for providing the necessary knowledge and academic support.

I would also like to thank my friends and classmates for their cooperation and suggestions. Finally, I am deeply grateful to my family for their constant motivation and encouragement, which helped me complete this project successfully.

Thank you to everyone who contributed directly or indirectly to the completion of this mini project.

Renulasya

24BAI70210

CERTIFICATE

This is to certify that the project report entitled **"Blog Publishing Platform (Inkly – A Community Blogging Website)"** is the original work carried out by **Renulasya, UID: 24BAI70210**, under my supervision, in partial fulfilment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering (AIT – AIML)** at Chandigarh University.

The work presented in this report is based on the research, design, implementation, and findings carried out by the student in accordance with the academic norms and guidelines of Chandigarh University. The report has been reviewed and found to meet the prescribed academic standards and project requirements.

_____

SIGNATURE

Ms. Lovleen Kaur (E20454)

Assistant Professor (APEX – CSE)

Date: _____

Table of Content

Chapter	Content	Page No.
	List of tables	
	List of figures	
	Abstract	
1.	**Introduction**	
	1.1 Background	
	1.2 problem statement	
	1.3 Proposed System	
	1.4 Scope of the Project	
2.	**Objectives**	
3.	**Software Requirements**	
4.	**Theory**	
	4.1 System Architecture	
	4.2 Main Modules	
	4.3 Data model	
	4.4 System Design	
5.	**Procedure**	
6.	**Code**	
7.	**Results**	
	7.1 Discover Dashboard	
	7.2 Story Publishing Interface	
	7.3 Article Reading and Comments	
	7.4 Bookmarks and Drafts	
	7.5 Search, Profile and Dark Mode	
8.	**Conclusion and Future Scope**	

List of Tables

Table No.	Description	Page No.
3.1	Hardware Requirements	
3.2	Software Requirements	
4.1	System Architecture	
4.5	Proposed Data Entities	
7.1	Functional Test Cases	

List of Figure

Figure No.	Description	Page No.
7.1	Inkly Discover Dashboard	
7.2	Story Publishing Interface	
7.3	Article Reading Page with Comments	
7.4	My Stories Page	
7.5	Bookmarks Interface	
7.6	Draft Management Page	
7.7	Search Results Interface	
7.8	Profile and Settings Interface	
7.9	Dark Mode Interface	

Abstract

The Blog Publishing Platform (Inkly) is a web-based application designed to provide an interactive and user-friendly environment for creating, publishing, managing, and discovering blog articles. The main aim of this project is to simplify the blogging process by providing a single platform where users can read articles, write their own stories, organize content, and interact with other readers.

The application is developed using React, Vite, HTML, CSS, and JavaScript. It provides features such as a Discover dashboard, user interface for story publishing, article search, category and tag filtering, bookmarks, likes, comments, draft management, profile settings, and dark mode. The interface is designed with a modern layout, attractive colors, interactive buttons, and responsive components to improve the user experience.

The current prototype uses browser local storage to retain selected content and user interactions. It demonstrates the frontend functionality and overall workflow of a blogging website. Backend integration using Node.js and Express, MongoDB database connectivity, secure user authentication, and administrator-based content moderation can be added in future development to make the platform a complete full-stack application.

This project provides practical knowledge of frontend development, component-based architecture, user interface design, and interactive web application development. It also establishes a foundation for building a scalable blogging platform that can support content creators and readers.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

CHAPTER 1

INTRODUCTION

1.1 Background

Blogging is a popular way to share knowledge, experiences, ideas, opinions, and information through the internet. It allows students, professionals, developers, and content creators to publish articles and communicate with readers. A blogging platform provides an organized environment where users can create, edit, publish, and manage their content.

The Blog Publishing Platform (Inkly) is a web-based application designed to provide a modern, interactive, and user-friendly blogging experience. It allows users to explore articles, search for topics, browse categories, save interesting stories, and interact with content through likes and comments.

The platform includes a visually attractive dashboard, story cards, category filters, search functionality, bookmarks, draft management, and a story publishing interface. It also provides profile settings and a dark mode option to improve the user experience.

The application is developed using React, Vite, HTML, CSS, and JavaScript. The current prototype uses browser local storage to retain selected content and interactions. The project provides practical experience in frontend development and establishes a foundation for future backend and database integration.

1.2 Problem Statement

Many basic blogging websites provide limited functionality for creating, organizing, searching, and managing articles. Users may experience difficulty managing unpublished drafts, finding relevant content, saving articles for later reading, and interacting with other readers.

A modern blogging platform should provide a simple interface that combines content creation, publishing, searching, categorization, and user interaction in one place.

Therefore, the problem addressed by this project is to develop an attractive and interactive blogging platform that simplifies article management and improves the overall experience for writers and readers.

The platform aims to organize blogging activities through a centralized dashboard and provide convenient access to important features such as article creation, search, bookmarks, comments, and draft management.

1.3 Proposed System

The proposed system, Inkly, is a web-based blogging platform that provides an interactive environment for discovering and managing articles. It is designed with a modern dashboard, clear navigation, colorful interface elements, and organized article cards.

The system provides the following features:

- Discover Dashboard: Displays featured stories and available articles.
- Story Publishing: Allows users to enter article titles, descriptions, categories, tags, and content.
- Search Functionality: Helps users find articles based on relevant keywords.
- Categories and Tags: Organizes articles according to their topics.
- My Stories: Provides access to a user's articles.
- Draft Management: Allows unfinished articles to be saved for later editing.
- Bookmarks: Allows users to save interesting articles.
- Likes and Comments: Demonstrates interaction with published content.
- Profile and Settings: Provides profile customization and appearance options.
- Dark Mode: Offers an alternative visual theme.
- Moderation Interface: Provides a foundation for an administrator content-review workflow.

The current implementation focuses on the frontend interface and browser-based interactions. Backend services, database integration, secure authentication, and complete administrator permissions can be incorporated in future development.

1.4 Scope of the Project

The scope of the Blog Publishing Platform is to design and develop a responsive, interactive, and visually appealing blogging website. The application focuses on providing a convenient interface for reading articles and managing blog content.

The project covers the design of the dashboard, article cards, story editor, search and category filters, bookmarks, likes, comments, drafts, profile settings, and dark mode. It also demonstrates how React components can be used to organize an interactive web application.

The current prototype stores selected data in the browser using local storage. It is suitable for demonstrating the user interface and basic frontend workflows.

The major areas covered by the project are:

- Development of a responsive blogging dashboard.
- Display of featured stories and article cards.
- Search functionality for finding articles.
- Category and tag-based content organization.
- Interface for creating, editing, and publishing stories.
- Draft management for unfinished articles.
- Bookmark and like functionality.
- Comment interface for reader interaction.
- Profile customization and dark mode.
- Navigation for personal stories and community content.

The current version is a frontend prototype that uses browser local storage for selected demo data. Real user authentication, shared database storage, secure file uploads, and server-side moderation are outside the current implementation and can be added in future versions.

Overall, this project provides a foundation for developing a complete blogging application that supports content creators and readers through an organized and interactive digital platform.

CHAPTER 2

OBJECTIVES

The main objective of the Blog Publishing Platform (Inkly) is to develop an interactive, attractive, and user-friendly web application that simplifies the process of creating, publishing, managing, and discovering blog articles.

The specific objectives of the project are as follows:

1. To Develop a User-Friendly Interface: To design a colorful, responsive, and easy-to-navigate interface that provides a smooth experience for readers and writers.
2. To Provide Article Publishing: To develop an interface that allows users to create, edit, publish, and manage blog articles.
3. To Implement Search Functionality: To enable users to search for articles using relevant keywords and find content easily.
4. To Organize Content: To categorize articles using categories and tags, making it easier for users to explore topics of interest.
5. To Support Draft Management: To allow users to save unfinished articles as drafts and continue editing them later.
6. To Provide Bookmark Functionality: To allow users to save interesting articles for future reference.
7. To Enable User Interaction: To provide features such as likes and comments to demonstrate interaction between readers and published content.
8. To Implement Appearance Settings: To provide profile customization options and dark mode to improve the user experience.
9. To Maintain Demo Data: To use browser local storage to retain selected stories and interactions across page refreshes.
10. To Provide a Foundation for Future Development: To establish a foundation for integrating user authentication, database connectivity, and administrator-based content moderation in future versions.

CHAPTER 3

SOFTWARE REQUIREMENTS

Software requirements describe the software, tools, and technologies needed to develop and run the **Blog Publishing Platform (Inkly)**. These requirements help ensure that the application can be developed, executed, and tested successfully.

3.1 Hardware Requirements

The hardware requirements specify the minimum computer resources needed to develop and run the application.

S. No.	Component	Requirement
1	Processor	Intel Core i3 or equivalent
2	RAM	Minimum 4 GB
3	Hard Disk	At least 500 MB of free space
4	Monitor	Standard display with 1366 × 768 resolution or higher
5	Keyboard and Mouse	Standard input devices
6	Internet Connection	Required for downloading dependencies and accessing online resources

Table 3.1. Hardware requirements

3.2 Software Requirements

The software requirements include the development environment, programming technologies, and browser needed to build and run the project.

S. No.	Software	Purpose
1	Operating System	Windows, Linux, or macOS
2	Visual Studio Code	Used to write, edit, and manage source code

3	HTML5	Provides the basic structure of web pages
4	CSS3	Used for styling, colors, layouts, and responsive design
5	JavaScript	Implements interactive functionality
6	React.js	Used to build reusable user interface components
7	Vite	Provides the development server and build tools
8	Node.js	Provides the JavaScript runtime environment
9	npm	Used to install and manage project dependencies
10	Lucide React	Provides icons for navigation and interface controls
11	Web Browser	Used to run and test the application
12	Browser localStorage	Stores demo content and selected interactions locally

Table 3.2 Software requirements

3.3 Technologies Used

1. HTML5

HTML5 provides the basic structure of the application, including page sections, headings, forms, buttons, and content areas.

2. CSS3

CSS3 is used to design the appearance of the website. It provides colors, typography, spacing, layouts, responsive styling, and dark mode.

3. JavaScript

JavaScript adds interactivity to the website, including search, filtering, navigation, and user interface actions.

4. React.js

React.js is used to create reusable components and manage the user interface. It helps organize features such as article cards, navigation menus, story editors, and profile settings.

5. Vite

Vite provides a fast development environment for running and building the React application.

CHAPTER 4

THEORY

4.1 Introduction

The Blog Publishing Platform, named **Inkly**, is a web-based application designed to provide an interactive environment for creating, publishing, managing, and discovering blog articles. The platform combines content creation and content discovery features within a single user interface.

The main purpose of the system is to simplify blogging activities by providing an organized dashboard, article cards, search functionality, categories, bookmarks, likes, comments, and draft management. The application uses modern frontend technologies to provide a visually attractive and user-friendly experience.

The current version is a frontend prototype developed using React, Vite, HTML, CSS, and JavaScript. It uses browser localStorage to retain selected demo content and interactions. Backend services and database integration can be added to support real users and shared content.

4.2 Working Principle of the System

The system works by displaying blog articles through a dashboard. Users can browse the available stories, search for specific topics, select categories, and open individual articles to read their content.

Writers can access the story editor and enter the title, description, category, tags, cover image URL, and article body. They can publish a completed story or save unfinished content as a draft.

The interface also provides interaction options such as liking articles, bookmarking stories, and adding comments. Selected demo data is stored in the browser so that it can remain available after refreshing the page.

The main working steps are:

1. The user opens the Inkly website in a web browser.
2. The Discover dashboard displays featured stories and article cards.
3. The user searches for articles or filters them by category.
4. The user opens a story to read its content.
5. The user can interact with available features, such as likes, bookmarks, and comments.
6. A writer can create a new story using the story editor.

4.3 System Architecture

The system architecture describes how the different parts of the application work together.

The current prototype consists of three main layers.

1. Presentation Layer

This layer contains the user interface, including the dashboard, navigation menu, article cards, story editor, reader view, and settings.

2. Application Layer

This layer manages interface interactions such as searching, filtering, creating stories, editing drafts, liking articles, bookmarking content, and displaying comments.

3. Browser Storage Layer

This layer uses localStorage to retain selected demo data in the user's browser.

A future full-stack version can include a Node.js and Express backend, secure authentication, and a MongoDB database to support persistent shared content.

Table 4.1 System Architecture of Inkly

React User Interface
↓
Application Logic and State Management
↓
Browser localStorage
↓
Future Enhancement: Node.js, Express and MongoDB

4.4 Main Modules of the System

4.4.1 Discover Module

The Discover module is the main dashboard of the application. It displays featured articles, story cards, categories, and navigation options. It helps users explore available content in an organized manner.

4.4.2 Search and Category Module

This module allows users to find articles by entering keywords. Category and tag filters help users narrow down the displayed content and discover articles related to their interests.

4.4.3 Story Publishing Module

The story publishing module provides an editor for entering article information. The writer can add a title, description, category, tags, cover image URL, and article body. The story can then be published or saved as a draft.

4.4.4 My Stories Module

This module provides access to the user's own stories. It supports the organization and management of content created through the demo interface.

4.4.5 Draft Management Module

The draft management module allows writers to save unfinished articles and return to them later. It helps organize content that is not yet ready for publication.

4.5 Data Model

A data model describes how information can be organized and stored in a database. For a future full-stack version of Inkly, the following entities can be used.

Entity	Description
User	Stores user profile and account information
Post	Stores article title, content, category, tags, and status
Comment	Stores comments associated with articles
Like	Records user likes on articles
Bookmark	Records articles saved by users
Moderation Log	Records administrative review decisions

Table 4.5 Data Model

These entities represent a proposed database design. The current frontend prototype uses browser localStorage rather than a connected MongoDB database.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

CHAPTER 5

PROCEDURE

5.1 Introduction

The procedure and methodology describe the steps followed during the development of the Blog Publishing Platform, named Inkly. The project focuses on designing an attractive, interactive, and user-friendly web application interface that allows users to discover articles, create blog posts, manage drafts, and interact with published content.

The development process begins with identifying the project requirements and planning the user interface. It then proceeds through environment setup, component development, styling, implementation of interactive features, browser storage, and testing. Each stage contributes to creating an organized and responsive blogging application.

The project uses React for building reusable user interface components, Vite for the development environment, HTML for the page structure, CSS for styling, and JavaScript for application logic. Browser localStorage is used to retain selected demo content and interactions.

The methodology follows a step-by-step development process so that the individual features can be developed and checked systematically. The current implementation focuses on frontend functionality. Backend integration, real authentication, and database connectivity are considered future enhancements.

5.2 Requirement Analysis

Requirement analysis is the first stage of the development process. It identifies the features required by the application and the needs of the intended users. For the Inkly platform, the main users are readers and writers, while administrators represent an additional role for future moderation functionality.

Readers need a convenient way to discover articles, search for topics, browse categories, read stories, and save interesting content. Writers need an interface for creating articles, editing existing posts, publishing content, and saving unfinished work as drafts.

The platform therefore requires a dashboard with navigation, article cards, search controls, category filters, and story management options. It also requires interactive features such as likes, bookmarks, comments, and appearance settings.

The main functional requirements identified for the project are:

1. Display available articles in an organized dashboard.

2. Provide search functionality for finding relevant stories.
3. Organize articles using categories and tags.
4. Provide a story editor for entering article details.
5. Support publishing and draft management workflows.
6. Allow users to save articles using bookmarks.
7. Demonstrate likes and comments on articles.
8. Provide navigation to personal stories and other sections.
9. Offer profile and appearance settings.
10. Provide a foundation for future administrator moderation.

The non-functional requirements include usability, responsive design, consistency, readability, maintainability, and reasonable performance in a modern web browser.

5.3 Planning and System Design

After identifying the requirements, the application structure and navigation flow are planned. The objective of the design stage is to create a consistent interface that allows users to move between the main features without confusion.

The application uses a dashboard layout. A sidebar groups the navigation options into workspace and community sections. The top navigation area contains search, appearance controls, notifications, and the option to write a story. The main content area displays featured content, category filters, and article cards.

The design uses a consistent color palette, readable typography, rounded cards, spacing, icons, and clear interactive controls. Purple accents distinguish important actions, while the content cards organize article information into readable sections.

The principal sections planned for the application are:

- Discover
- My Stories
- Bookmarks
- Drafts
- Trending
- Moderation
- Settings

The layout is designed to adapt to different screen sizes. On larger screens, the sidebar and article cards provide a spacious dashboard. On smaller screens, the content layout should adjust to make navigation and article information easier to view.

During planning, the data needed for each article is also identified. A typical article may contain a title, description, body, category, tags, cover image, author information, publication date, and status.

5.4 Development Environment Setup

The next step is to prepare the software environment required for development. Visual Studio Code is used as the code editor, while Node.js provides the JavaScript runtime required by the development tools. npm is used to install project dependencies and execute scripts.

The project is organized using Vite, which provides a development server and a frontend build process. React is used to develop the interface through reusable components.

The general procedure for setting up and running the application is as follows:

Step 1: Install Node.js on the computer. npm is included with the standard Node.js installation.

Step 2: Open Visual Studio Code and select the project folder using the Open Folder option.

Step 3: Open the integrated terminal by selecting Terminal and then New Terminal.

Step 4: Install the project dependencies using the following command:

npm install

This command reads the project's package configuration and installs the required packages.

Step 5: Start the development server:

npm run dev

Step 6: Open the local development URL displayed in the terminal. For a typical Vite configuration, the address is http://localhost:5173, although the actual port may differ.

Step 7: Check that the dashboard loads in the browser and that the navigation and interface elements are visible.

This environment setup allows the developer to modify the source code and check the updated interface in the browser.

5.5.1 Page Layout Development

The main page layout is developed first. It includes the sidebar, top navigation bar, content area, and footer. The sidebar provides access to the major sections of the application, while the top navigation area provides quick access to search and story creation.

The main content area is organized to display featured stories and article cards. Each card presents information such as the article title, category, author, date, and engagement details where available.

Reusable layout elements provide a consistent appearance across the different sections of the application.

5.5.2 Component Development

The interface is divided into reusable React components. This approach makes the application easier to maintain because each component has a specific responsibility.

Examples of component responsibilities include:

- Navigation components display the available sections.
- Story cards present article summaries.
- Search controls accept keywords.
- Category controls allow topic filtering.
- Editor components collect article details.
- Modal components display forms and settings.
- Reader components display article content and comments.

React state is used to control interactive elements and update the interface when the user performs an action.

5.5.3 Styling and Responsive Design

CSS is used to create the visual appearance of the platform. It defines colors, typography, spacing, borders, shadows, card layouts, menus, and buttons.

The interface uses consistent styling to make the dashboard look organized. Buttons are designed to indicate their purpose, while spacing and visual grouping help users distinguish different sections.

Responsive styling allows the layout to adjust to different screen widths. This improves usability when the application is viewed on a laptop, desktop, tablet, or mobile browser.

5.6 Implementation of Main Features

After creating the basic layout, the major application features are implemented and connected to the user interface.

5.6.1 Discover Dashboard

The Discover dashboard acts as the main entry point for browsing articles. It displays featured content and a collection of article cards.

Category filters help users narrow the available content. Each card provides a summary so that readers can decide which article they want to open.

The dashboard combines navigation and content discovery in a single interface.

5.6.2 Search and Category Filtering

The search feature helps users locate articles by entering relevant keywords. The application compares the search query with the available article information, such as titles, descriptions, categories, or tags, depending on the implemented search logic.

Category filtering allows users to select a topic and view relevant articles. Search and category selection improve content discovery by reducing the number of unrelated results displayed.

For example, a user searching for a technology-related topic should see matching stories rather than the entire article collection.

5.6.3 Story Publishing

The story publishing module provides a form through which writers can enter article details. The fields may include the title, description, category, tags, cover image URL, and article body.

The publishing workflow involves entering the required information, checking the form, and selecting the appropriate action.

The general publishing procedure is:

1. Open the Write a Story option.
2. Enter the article title.
3. Add a short description.
4. Select a category.
5. Enter relevant tags.
6. Add the main article content.

7. Provide a cover image URL if supported.
8. Review the entered information.
9. Publish the article or save it as a draft.

The application should check required fields before accepting a submission. A complete publishing workflow should provide feedback when the story is saved or published.

5.6.4 My Stories

The My Stories section provides access to the user's own articles. It supports the content management workflow by grouping personal content in one place.

Depending on the available implementation, users can open their articles and access editing or management options. This section makes it easier to return to previously created content without searching the entire community feed.

5.6.5 Draft Management

Draft management is intended for articles that are not yet ready for publication. Writers can save incomplete content and return to it later.

The draft workflow involves entering article details, selecting the save-draft action, and opening the Drafts section to retrieve the saved article.

Draft status distinguishes unfinished content from published articles. A draft should remain unpublished until the writer explicitly completes the publishing process.

5.6.6 Bookmarks and Likes

Bookmarks allow readers to save articles for later reading. When a reader bookmarks a story, the application records the selected story and makes it accessible through the Bookmarks section.

The like feature allows users to express appreciation for an article. The interface may update the displayed like count or visual state after the action.

These features demonstrate common interactions found on content publishing platforms. In the current prototype, their persistence depends on the implemented browser storage and state-management logic.

5.6.7 Comments

The comments feature allows readers to provide feedback on articles. The interface can display existing comments and provide a text field for entering a new comment.

The general procedure involves opening an article, entering a comment, submitting it, and checking whether the comment appears in the relevant section.

5.6.8 Profile, Settings, and Dark Mode

The profile menu provides access to personal story management and other user-related actions. The settings interface provides options for customizing the demo experience.

Dark mode changes the visual theme of the application. The interface should update the background, text, borders, and other relevant elements to maintain readability.

These features improve the flexibility of the interface and demonstrate how user preferences can be incorporated into a web application.

5.6.9 Moderation Interface

The moderation section represents the planned administrative workflow for reviewing submitted content.

A complete moderation system would allow authorized administrators to inspect submitted articles, approve suitable content, reject inappropriate submissions, and record moderation decisions.

The current prototype should only be described as implementing these actions if they are actually connected and tested. Secure administrative permissions require backend authorization and cannot be enforced reliably through frontend controls alone.

5.7 Browser Storage and Data Management

The current prototype uses browser localStorage to retain selected demo content and interactions. This allows information to remain available after refreshing the page in the same browser profile.

The storage mechanism works by converting JavaScript data into a JSON string before saving it. When the application needs to restore the information, the saved string can be converted back into a JavaScript value.

The following is an illustrative example of storing an array of posts:

```
localStorage.setItem(
  "inkly-posts",
  JSON.stringify(posts)
);
```

The corresponding data can be retrieved using:

```
const savedPosts = JSON.parse(
  localStorage.getItem("inkly-posts") || "[]"
```

);

These examples explain the general browser storage technique. The actual storage keys and code should match the project's source files.

Browser storage is suitable for demonstrating local interactions, but it does not provide shared access across different users or devices. It should not be used to store passwords or other sensitive authentication information.

For a complete full-stack application, the frontend can communicate with a Node.js and Express backend. MongoDB can store users, articles, comments, likes, bookmarks, and moderation records.

5.8 Testing and Verification

Testing is performed to check whether the main interface elements and user workflows behave as expected.

The application should be opened in a browser after starting the development server. Each major feature can then be checked individually.

The testing procedure includes the following activities:

1. Open the Discover dashboard and verify that the page loads.
2. Navigate between the available sections.
3. Enter a search term and inspect the displayed results.
4. Select a category and verify the filtered content.
5. Open the story editor and enter article details.
6. Save a draft and check the Drafts section.
7. Publish a test article and check the relevant story view.
8. Bookmark an article and inspect the Bookmarks section.
9. Test the like and comment interactions.
10. Refresh the browser and check the persistence of saved demo data.
11. Toggle dark mode and inspect the interface.
12. Resize the browser window and check the responsive layout.

CHAPTER 6
CODE

Application Entry Point

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles.css";


createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Story Data Structure

```
const samplePost = {
  id: 1,
  title: "Getting Started with Web Development",
  excerpt: "Learn the basics of modern web development.",
  author: "Renu Lasya",
  category: "Technology",
  tags: ["Web Development", "React"],
  content: "Web development involves creating websites...",
  status: "Published",
  likes: 0,
  comments: []
};
```

Displaying Blog Posts

```
function BlogCard({ post }) {
  return (
    <article className="blog-card">
      <span className="category">
        {post.category}
      </span>
      <h2>{post.title}</h2>
      <p>{post.excerpt}</p>
      <small>By {post.author}</small>
      <div className="blog-actions">
        <span>❤️ {post.likes}</span>
        <span>💬 {post.comments.length}</span>
      </div>
    </article>
  );
}
export default BlogCard;
```

Creating a New Blog Post

```
import { useState } from "react";
function CreatePost() {
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [message, setMessage] = useState("");
  function publishPost(event) {
    event.preventDefault();
    if (!title.trim() || !content.trim()) {
      setMessage("Enter a title and article content.");
      return;
    }
    const post = {
      id: Date.now(),
      title,
      content,
      status: "Published"
    };
    console.log("New post:", post);
    setMessage("Post details are ready to publish.");
  }
  return (
    <form onSubmit={publishPost}>
      <h2>Write a Story</h2>
      <input
```

CHAPTER 7

RESULTS

7.1 Introduction

The Blog Publishing Platform, named Inkly, was developed to demonstrate a modern, interactive, and user-friendly blogging website interface. The application provides an organized dashboard through which users can discover articles, explore categories, search for content, and access story management features.

The user interface is designed using React, Vite, HTML, CSS, and JavaScript. It uses a colorful theme, organized navigation, article cards, and interactive controls to improve the overall user experience.

The project demonstrates the frontend workflow of a blogging platform, including story publishing, draft management, bookmarks, likes, comments, profile settings, and dark mode. The current prototype uses browser localStorage to retain selected demo content and interactions.

7.2 Discover Dashboard-The Discover dashboard is the main screen of the application. It provides access to the available stories and the major sections of the platform. The dashboard contains navigation options, a search bar, category filters, featured content, and article cards.

The sidebar allows users to navigate to sections such as My Stories, Bookmarks, Drafts, Trending, and Settings. The article cards present story information in an organized format, making it easier for users to explore the available content.

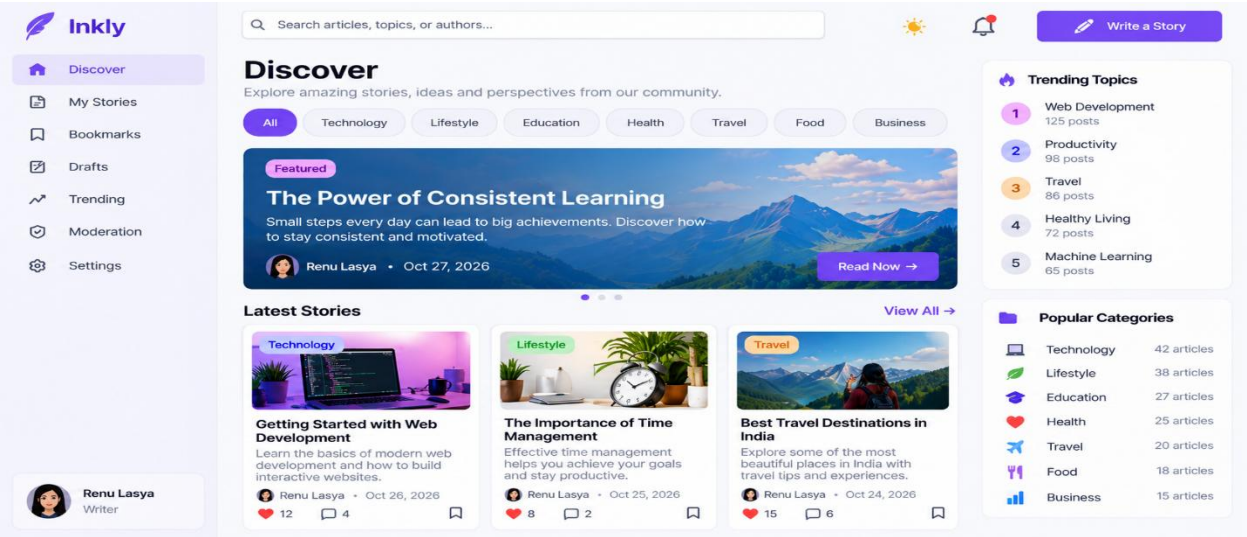


Fig. 7.1 Inkly Discover Dashboard

7.3 Story Publishing Interface-The story publishing interface provides a form for entering the details of a new blog article. The writer can enter the title, description, category, tags, cover image URL, and main article content, depending on the fields available in the application.

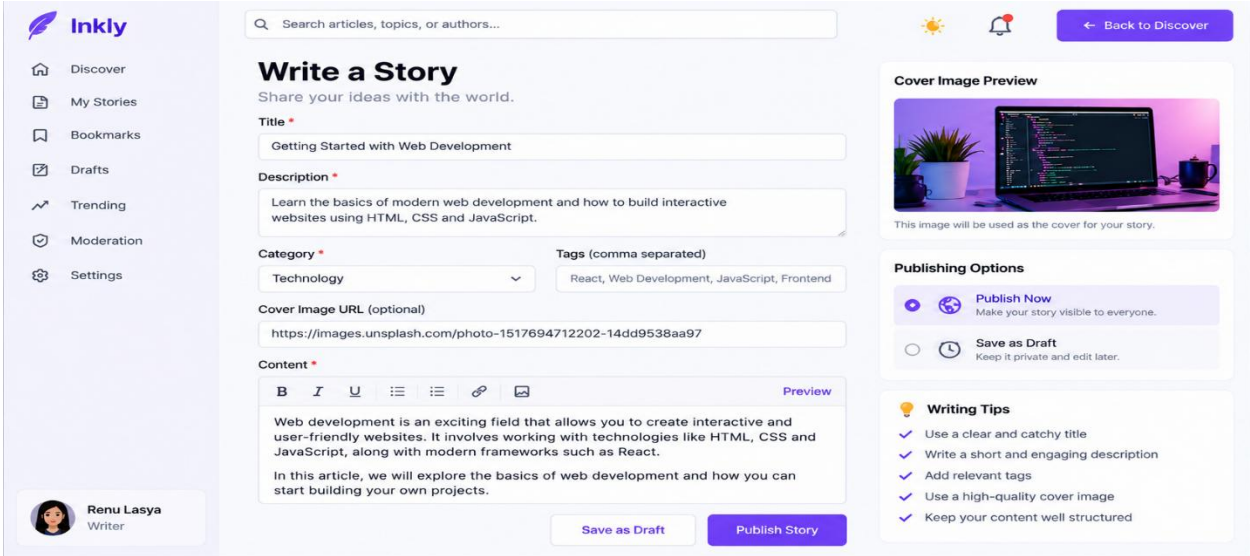


Fig. 7.2 Story Publishing Interface

7.4 Article Reading and Comments-The article reading interface presents the selected blog post to the reader. It provides a dedicated area for viewing the article content and related information. The comment section demonstrates how readers can share feedback about a story.

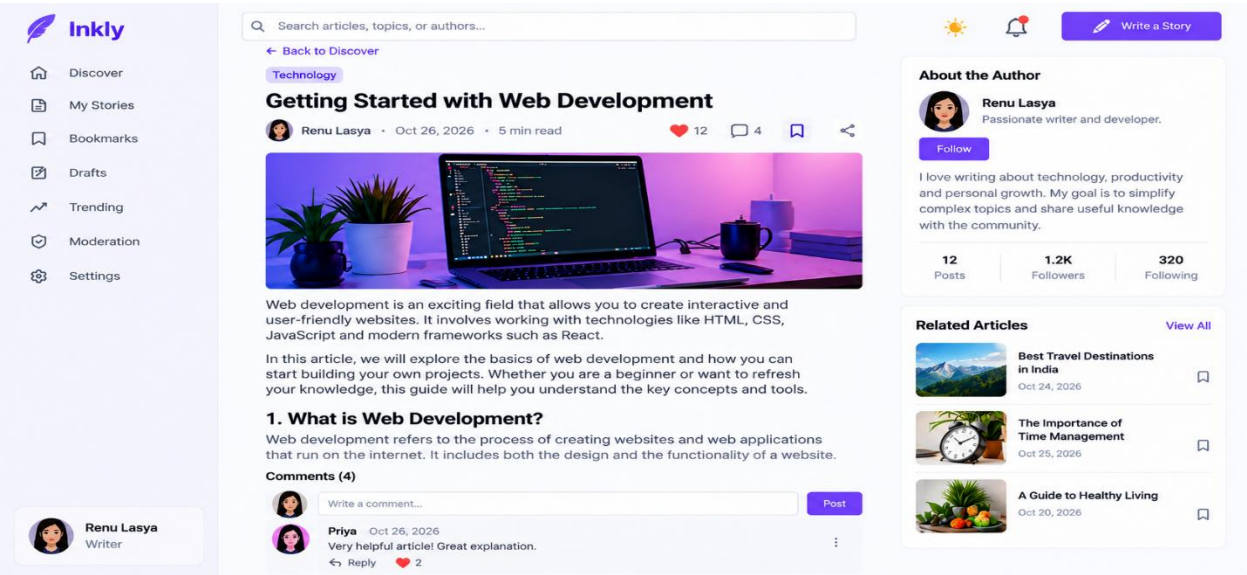


Fig. 7.3 Article Reading Page with Comments

7.5 My Stories Interface-The My Stories section is intended to help writers access and manage their own articles. It provides a dedicated area for viewing personal content without searching the entire collection of stories.

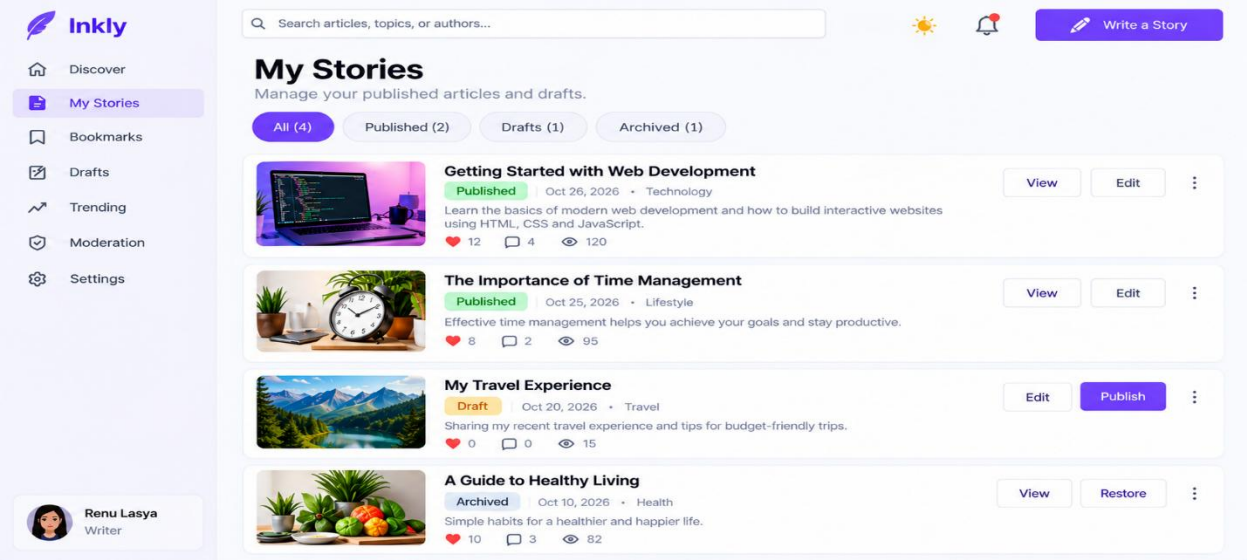


Fig. 7.4 My Stories Interface

7.6 Bookmarks Interface-The bookmarks feature allows users to save interesting articles for later reading. The Bookmarks section provides access to the stories selected by the user.

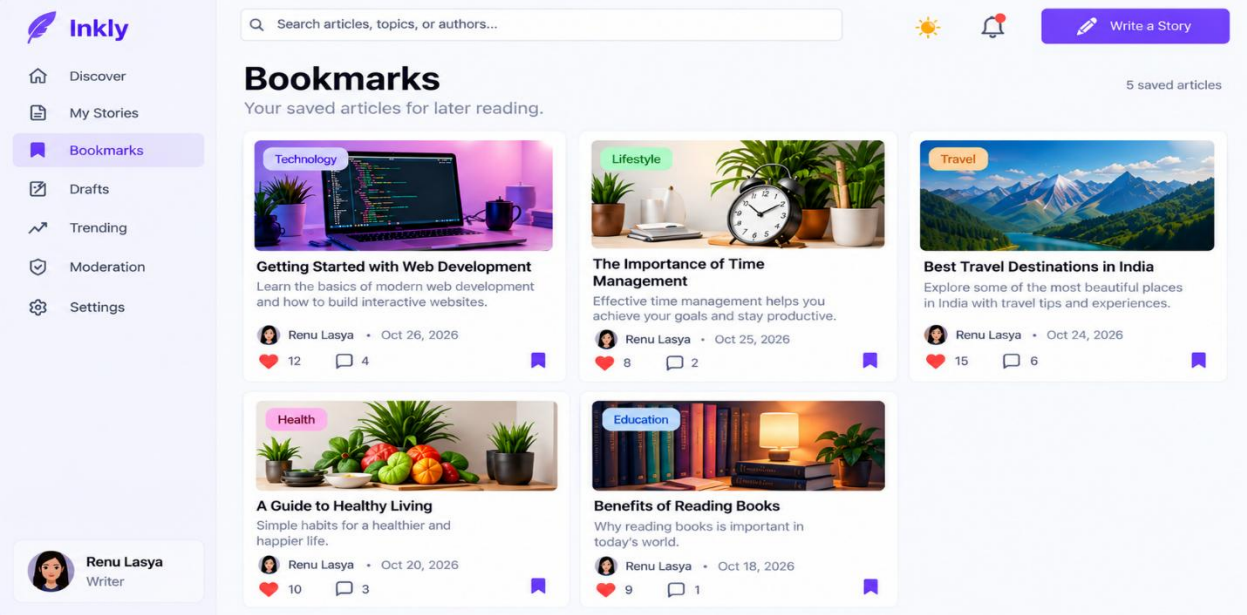


Fig. 7.5 Bookmarks Interface

7.7 Draft Management Interface- The Drafts section is designed for storing unfinished articles so that writers can return to them later. Draft management separates unpublished content from completed articles.

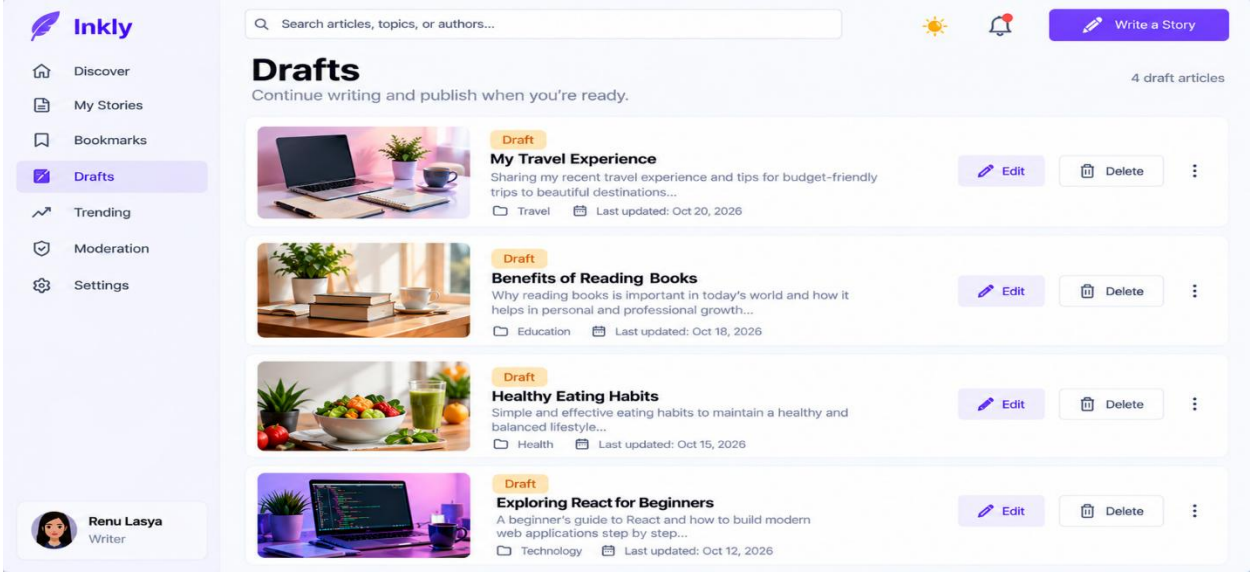


Fig. 7.6 Draft Management Interface

7.8 Search and Category Filtering-The search feature helps users locate relevant articles by entering keywords. Category filters allow users to narrow the displayed content according to the selected topic.

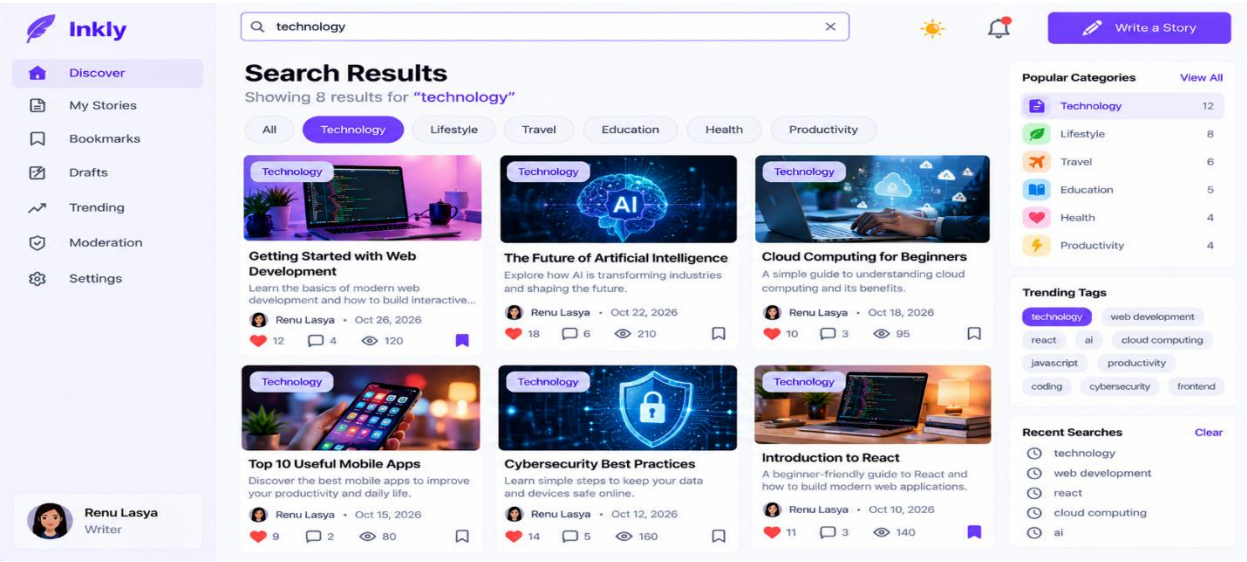


Fig. 7.7 Search and Category Filtering

7.9 Profile and Settings- The profile menu and settings interface provide access to user-related options and appearance preferences. These controls help organize the interface and allow users to customize selected aspects of the demo experience.

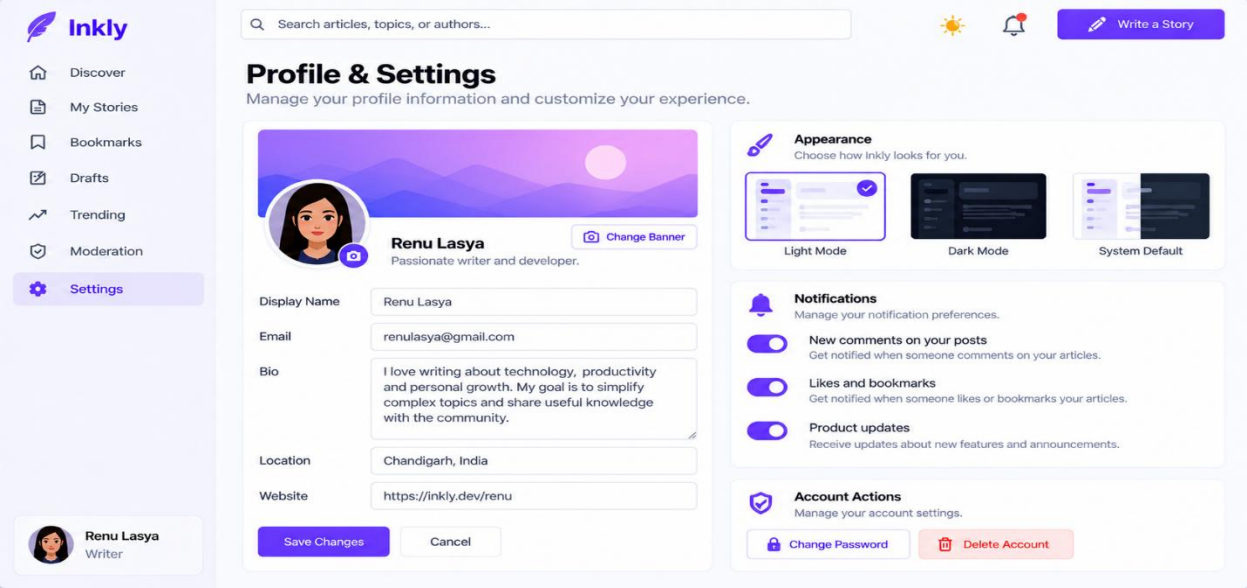


Fig. 7.8 Profile and Settings Interface

7.10 Dark Mode Interface- Dark mode provides an alternative appearance for the application. It changes the visual theme to use darker backgrounds and contrasting text and interface elements.

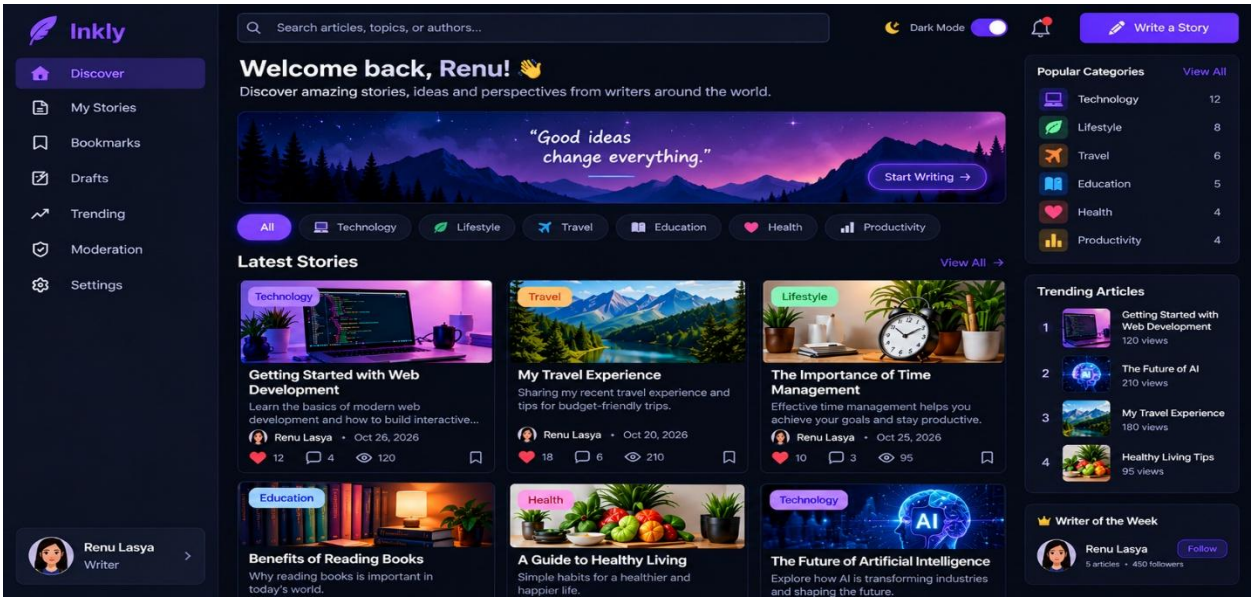


Fig. 7.9 Dark Mode Interface

7.11 Testing Results

The application should be tested in the browser to verify the main user workflows. The following table lists the expected results for the principal test cases. Mark each test as passed only after verifying it in the running project.

Test ID	Test case	Expected result
TC-01	Open the application	Dashboard loads correctly
TC-02	Search for an article	Matching stories are displayed
TC-03	Select a category	Relevant stories are filtered
TC-04	Create a blog post	Entered article details are accepted
TC-05	Save a draft	Draft is available in the Drafts section
TC-06	Bookmark an article	Article appears in Bookmarks
TC-07	Like an article	Like state or count updates
TC-08	Add a comment	Comment appears in the article view
TC-09	Refresh the browser	Saved demo data remains available where implemented
TC-10	Enable dark mode	Interface appearance changes

Table 7.1 Functional Testing Results

7.12 Overall Result Table

The Inkly project demonstrates the design of an interactive blogging platform with a modern dashboard and a range of content discovery and management interfaces. Its main purpose is to organize the experience of reading, creating, and managing blog articles within a single application.

The prototype provides a foundation for further development and can be extended with backend APIs, MongoDB database integration, secure user authentication, and role-based administrator moderation. These enhancements would help transform the frontend demonstration into a complete full-stack blogging platform.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

The Inkly – Blog Publishing Platform has several opportunities for future improvements. The current version provides a frontend prototype with features such as story publishing, drafts, bookmarks, search, categories, comments, and profile settings. In the future, the platform can be enhanced with the following features:

1. User Authentication: Implement secure registration and login using JWT authentication and password encryption.

2. Database Integration: Integrate MongoDB to permanently store user details, blog posts, comments, likes, and bookmarks.

3. Image and File Upload: Allow users to upload cover images and other media files directly from their devices.

4. Advanced Search: Improve search functionality with keyword matching, filters, tags, and author-based search.

5. Admin Dashboard: Develop a secure admin panel to manage users, review reported posts, and moderate inappropriate content.

6. Real-Time Notifications: Implement notifications for new comments, likes, replies, and other user activities.

7. Social Media Integration: Allow users to share their published articles through social media platforms.

8. Responsive Design: Further optimize the interface for mobile phones, tablets, laptops, and desktop devices.

9. Analytics Dashboard: Provide writers with statistics such as article views, likes, comments, and reader engagement.

10. Cloud Deployment: Deploy the application using cloud hosting services so that users can access the platform from anywhere.

These improvements can make Inkly a more secure, scalable, interactive, and user-friendly blogging platform in the future.